סיכום תרגולים 2027 – שפות תכנות

© גיא יער-און

ייתכן שנפלו טעויות בעת כתיבת הסיכום, ולכן השימוש בסיכום על אחריותכם בלבד.

תוכן עניינים:

תרגול 1: prolog 2

תרגול 1

שפות תכנות מחולקות לשני סוגים גדולים:

1. **שפות הצהרתיות (declarative):** בשפות אלו המיקוד הוא על מה המחשב צריך לעשות. השפה עוסקת בהצהרה על אובייקטים. למשל: CSS,HTML,SQL.

גם שפות הצהרתיות מתחלקות לשני סוגים:

- שפות בהן פונקציות קוראות לפונקציות – דוגמת Ocmal.
- Logic: כאן ישנם חוקים לוגיים וקשרים. למשל – Prolog עליה נדבר היום.

2. שפות מצוות (imperative): המיקוד כאן הוא איך המחשב צריך לעשות. השפה עוסקות בפקודות למערכת, למשל Java,python,C,C++ וכו'.

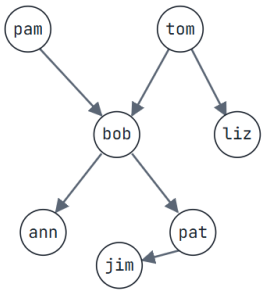
שפות מצוות מתחלקות לשלושה חלקים:

- **Von Neumann:** המודל החישובי של ון ניומן, מודל חישובי של שינוי משתנים.
- **Object-Oriented**: מודל מובנה של חישוב אובייקטים והאינטראקציה בניהם.
- **Scripting languages:** שפות סקריפטים, תיאום בין רכיבים.

כלומר, בC אנו כותבים אלגוריתם כמו לולאה, תנאי והשמה. בפרולוג נכתוב ידע – עובדות: "טום הוא ההורה של בוב" וחוקים: " סבא הוא הורה של הורה". ואז – נשאל שאלות. מנוע הפרולוג מחפש בעצמו את הדרך להוכיח את התשובה.

Prolog

על מנת להבין כיצד מתכנתים בפרולוג, נתבונן בדוגמה של עץ משפחה:



בשלב הראשון, נרצה לקבוע עובדות: למשל, ליז היא הבת של טום.

```
?- assert(parent(pam, bob)).
true.
?- assert(parent(tom, bob)).
true.
?- assert(parent(tom, liz)).
true.
?- assert(parent(bob, ann)).
true.
?- assert(parent(bob, pat)).
true.
?- assert(parent(pt, jim)).
true.
```

ואז, ניתן לכתוב שאילתה. למשל – האם ליז היא ההורה של PAT?

והתשובה -

```
?- parent(bob, pat).
true.
?- parent(liz, pat).
false.
```

כמובן – parent איננה פקודה בפרולוג. זה שם שהמצאנו. פרולוג לא יודע מה זה הורה, יכולנו לקרוא לזה גם בננה. מה שאומרים לפרו-לוג, יש לנו קשר בשם parent בין בוב לpat. זה הכל.

Assert: פקודה מובנית בשפה, באמצעותה מוסיפים לזיכרון. פרולוג עונה true: "הוספתי את המידע".

לאחר שהעובדות כבר בזיכרון, ניתן לשאול באמצעות הוספת "?" בתחילת השורה משמאל ולקבל true or false.

- בפרולוג כל פקודה מסתיימת בנקודה. ללא הנקודה פרולוג לא מריץ את השאילתא.
- בפרולוג – אנחנו מגדירים פרדיקטים (יחסים) כעובדות.
- שאילתא (Query), בודקת אם היחס המבוקש נמצא בפרדיקט שהגדרנו.

הכנסת עובדות לבסיס הנתונים:

פרט לassert, ישנה דרך נוספת לגרום לסביבת העבודה להכיר פרויקט: באמצעות טעינה מקובץ. כותבים את כל העובדות בקובץ עם סיומת pl. וטוענים אותו עם consult(<name of file without .pl>)

לדוגמה:

file.pl

```
parent(pam, bob).
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).
```

```
?- consult(file).
true.

?- parent(bob, pat).
true.

?- parent(bob, or).
false.
```

- בסיס הנתונים בו שמורות כל העובדות נקרא logicbase.

פרדיקטים

פרדיקט מוגדר ע"י השם ומספר הארגומנטים שלו. למשל:

$$\text{pred}(\text{arg1}, \text{arg2}, \ldots, \text{argN})$$

- שני פרדיקטים שונים עם אותו השם אך מס' ארגומנטים שונה – נחשבים לזוג פרדיקטים שונים.
- פרדיקט יכול להיות עם ארגומנט אחד בודד. זה משמש לרוב לציון עובדות בlogicbase.
- פרדיקט ללא ארגומנטים נכתב כך: .pred

ישנם מספר סוגי פרדיקטים:

סוג	מה זה	דוגמאות
מספר	שלם או ממשי	`28` `3.14`
אטום	קבוע: טקסט שמתחיל באות קטנה, או טקסט בין גרשיים `'…'`	`bob` `'Bar-Ilan'`
משתנה	מתחיל באות גדולה או ב-`_` (קו תחתון לבד = משתנה אנונימי)	`X` `Name` `_`
מבנה	נלמד בהמשך על סוגי מבנים בפרולוג	`point(1,2)`

- חשוב לזכור כי אות קטנה הוא ערך קבוע, ואות גדולה הוא משתנה שפרולוג ימלא עבורך.

פרדיקטים ופונקציות: בפרולוג משתמשים בפרדיקטים ופונקציות (שמבחינת מימוש נראות כמו פרדיקטים) בשביל לייצג עובדות בlogicbase, בשביל ייצוג של מבני נתונים וכן לייצג פונקציות לחישובים שונים.

- חשוב: בפרולוג אין return, פונקציה היא פרדיקט שאחד הארגומנטים שלה הוא **הפלט**. במקום len=size(list) נכתוב size(List, Len). הפרולוג ימלא את Len.

<u>דוגמה:</u>

```
parent(tom, bob).                  % עובדה
location(apple, kitchen).          % עובדה
costumer("Zvi", 28, 'Bar-Ilan').   % מבנה נתונים
size([1,2,3], N).                  % "פונקציה": N 3 יקבל
```

בדוגמה האחרונה – פרולוג רואה שN מתחיל באות גדולה לכן מבין שהוא משתנה שיש למלא. נשים לב שכרגע size לא הוגדרה לכן זה יחזיר שגיאה – בהמשך נטפל בזה.

פרדיקטים עם משתנים: ניתן לשלוח שאילתות עם משתנים על מנת לקבל מידע. למשל:

```
?- parent(X, liz).
X = tom.

?- parent(bob, X).
X = ann ;
X = pat.
```

```
logicbase
parent(pam, bob).
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).
```

נשים לב כי מקבלים רק את התשובה הראשונה, לפי הסדר שנכנס לlogicbase. לחיצה על נקודה פסיק תביא את התשובה הבאה – אם ישנה.

פרולוג עובר על העובדות לפי הסדר בקובץ ומחפש עובדה שניתן להתאים לשאילתה. ההתאמה הנ"ל נקרא **יוניפיקציה**. המשתנה X מקבל את הערך שגורם לשני הצדדים להיות זהים.

כמו כן, ניתן לשים משתנים בשני הארגומנטים. ושוב עם נקודה פסיק לקבל את כל המידע:

```
?- parent(X, Y).
X = pam,
Y = bob ;
X = tom,
Y = bob ;
X = tom,
Y = liz ;
```

- פרדיקט על משתנה: ניתן להגדיר פרדיקט על משתנה. המשמעות היא מעניינת – כל ארגומנט שיתקבל יקיים את הפרדיקט. נשים לב כי משתנה בעובדה הוא כמו $\forall x$ בלוגיקה. לכן כל שאילתא כאן בדוגמה מסוג sleep(bla-bla) מצליחה:

```
?- assert(sleep(X)).
true.

?- sleep(Zvi).
true.

?- sleep(students).
true.
```

שאילתות עם האופרטור וגם: ע"י הוספת פסיק ניתן לבצע שאילתות מהצורה וגם. למשל – בהינתן מאגר המידע שעליו אנחנו עובדים, שאילתא כמו:

```
?- parent(Y,jim), parent(X,Y).
Y = pat,
X = bob.
```

היא – מי הוא אבא של ג'ים (Y) וגם X שהוא אבא של Y? פרולוג הולך מחפש ומוצא: pat, bob. כיצד בפועל פרולוג פותר זאת? הוא מתחיל במטרה השמאלית – היא נותנת Y=pat. ואז המטרה השמאלית הופכת להיות $parent(Pat, X)$ שעבורה פרולוג מחזיר את BOB.

שאילתות עם האופרטור או: כאן יש להוסיף נקודה פסיק. למשל:

```
?- parent(X,jim); parent(X,ann).
X = pat ;
X = bob.
```

ביצענו את השאילתא $parent(X, jim)$ or $parent(X, ann)$. כלומר – תביא לי את ההורה של ג'ים או אן. מבחינת הסדר שפרולוג מחזיר – קודם את תשובות הצד השמאלי ורק אז את תשובות הצד הימני בשאילתה.

הערות: ניתן לכתוב בפרולוג הערות, הערת שורה מתחילה בסימון % והערה ארוכה כך:

/* your comment ....*/

- Write("Hello world") הוא פרדיקט שמדפיס.
- Nl – עושה זאת בשורה חדשה.
- Halt: עוצר את התוכנית.

```
?- write('hello world'), nl, halt.
hello world
zvi@com169433:~$
```

נשים לב שיש פסיקים בדוגמה - כי זה פשוט "וגם". פרולוג מריץ משמאל לימין – מדפיס, יורד שורה ויוצא.

מבנה: ניתן להשתמש בפונקציות כייצוג של מבנה אשר מחבר כמה פרמטרים יחד. למשל:

```
?- assert(costumer("Zvi", 28, Bar-Ilan)).
true.

?- costumer(X, 28, Bar-Ilan).
X = "Zvi".
```

כאן הגדרנו פונקציה באופן רגיל. "המבנה" הוא בצורת התפיסה שלנו. נשים לב כי שאלנו על X כמשתנה, קיבלנו צבי. ניתן לשאול ע"י השמת משתנה במקום אשר רוצים לגלות ולקבל מי מקיים את הארגומנט, שאיננו משתנה.

<u>דוגמה:</u> נתבונן על קובץ הpl הבא –

door.pl

```
room(kitchen).
room(office).
room(hall).

door(office, hall).
door(kitchen, office).

location(desk, office).
location(apple, kitchen).
location(computer, office).
```

```
?- room(X).
X = kitchen ;
X = office ;
X = hall.
```

הפרדיקט ROOM מגדיר כי ביחס 3 איברים. לכן ניתן לבצע שאילתא עם משתנה X ועם נקודה פסיק לקבל את כל מי שבחדר. פעולה זו נקראת **Binding:** קישור משתנה. המשתנה X נקשר לערך, בכל פעם שלוחצים נקודה פסיק פרולוג משחרר את הקישור הקודם ומנסה את העובדה הבא. הסדר חשוב בקישור!

- הבעיה: נשים לב כי הגדרנו door(office, hall) ולכן door(hall, office) יכשל. **אבל דלת היא סימטרית!** פרולוג לא יודע זאת כמובן. הפתרון הנאיבי הוא לכתוב כל דלת פעמיים. אבל יש פתרון טוב יותר: יש מעבר מX לY אם יש דלת מX לY או מY לX.

חוקים: עובדות נתנו לנו דרך לתאר יחסים ועובדות. למשל המשרד הוא חדר .room(office) מלבד עובדות, ניתן להגדיר בפרולוג חוקים – חוקים מאפשרים לכתוב משפטים מהסוג "אם X מתקיים, אז Y מתקיים".

התחביר	`head :- body.`
מתפרש כ-	`if body then head.`

כיצד קוראים את זה? את :- קוראים כמו חץ שמאלי ← הראש הוא המסקנה, והגוף הוא התנאי. הסדר הפוך מif רגיל. למשל:

grandfather(X, Z): -parent(X, Y), parent(Y, Z)

אומר כי X סבא של Z אם קיים Y – הורה, כך שX הורה של Y וגם Y הורה של Z.

כעת כיצד מממשים דלת דו צדדית? בקלות:

```
connect(X,Y) :- door(X,Y).
connect(X,Y) :- door(Y,X).
```

המשמעות היא שהדרך להתחבר בין X לY היא אם קיימת דלת מX לY, ואפשרות נוסף אם קיימת מY לX.

חוקים – סמנטיקה

בהינתן החוק P: -Q, R. ניתן לומר כי:

#	היגד	סוג
1	P אמת רק אם Q וגם R אמת.	**משמעות הצהרתית** `declarative meaning`
2	מ-Q ומ-R נובע P.	
3	כדי לפתור את הבעיה P, יש לפתור קודם את תת-הבעיה Q ואחר כך את תת-הבעיה R.	**משמעות פרוצדורלית** `procedural meaning`
4	כדי לספק את P, יש לספק קודם את Q ואחר כך את R.	

הסוג הצהרתית משמע מה נכון, לוגיקה טהורה. סדר איננו משנה. פרוצדורלית – כיצד פרולוג מחושב בפועל. כאן הסדר כן משנה: פרולוג תמיד מנסה משמאל לימין, ומלמעלה למטה. מדוע זה חשוב? שתי תוכניות יכולות להיות שקולות לוגית אך אחת תעבוד והשנייה תכנס ללולאה או תיתן שגיאה בגלל סדר המטרות.

אריתמטיקה:

```
?- X = 1 + 2.
X = 1+2.

?- X is 1 + 2.
X = 3.
```

לפרולוג הוסיפו את האפשרות לחשב ביטויים אריתמטיים. אך כיוון שהשפה עובדת באמצעות השוואת תבניות יש לכתוב במפורש שרוצים לחשב את הביטוי עם האופרטור is. האופרטורים הנתמכים הם -,+,*,/,mod. דוגמה:

Is מחשב את הצד הימני ומתאים את התוצאה לצד השמאלי. הצד הימני של is חייב להיות ידוע לגמרי, אחרת שגיאה. כמו כן פרולוג תומכת באופרטורים שמשווים בין שני ביטויים אריתמטיים:

אופרטור	משמעות
`X > Y`	גדול
`X < Y`	קטן
`X >= Y`	גדול או שווה
`X =< Y`	קטן או שווה (שימו לב לכתיב!)
`X =:= Y`	הערכים של X ו-Y זהים
`X =\= Y`	הערכים של X ו-Y שונים

אופרטורי השוואה מחשבים את שני הצדדים לפני ההשוואה! בניגוד לis למשל שמחשב רק את צד ימין. קטן או שווה נראה כמו מתמטיקה, ולא <=. זה נראה כמו חץ – והוא שמור.

- האופרטור = משווה בין מחרוזות. לעומת זאת, האופרטור X=:=Y מחשב את X ואת Y ומחזיר אם הם שווים.

```
?- 1 + 2 = 2 + 1.
false.
```

```
?- 1 + 2 =:= 2 + 1.
true.
```

<u>דוגמה:</u> פרדיקט שמחשב $gcd(a,b)$. המחלק המשותף הגדול ביותר של זוג מספרים , ידוע כי עבור a>b מתקיים $gcd(a,b) = gcd(a-b,b)$ ועל סמך הטענה הזו נציג אלגוריתם בפרולוג. למעשה, הגדרנו שלושה חוקים:

```
gcd.pl
gcd(X, X, X).
gcd(X, Y, D) :-
    X < Y,
    Y1 is Y - X,
    gcd(X, Y1, D).
gcd(X, Y, D) :-
    X > Y,
    gcd(Y, X, D).
```

1. Gcd של X עם עצמו – הוא X.
2. אם X<Y אזי מחסירים את הקטן מהגדול, כלומר מחליפים את המשתנה Y להיות $Y1 = Y-X$.
3. אחרת, בדומה.

למעשה זה קוד רקורסיבי, שמורץ שוב ושוב עד שמקבלים את X עם עצמו. השלישי משמאל בכלל הראשון הוא X – שמחזירים. לכן איך נריץ את זה? נריץ $gcd(7,12,S)$ על מנת לקבל לתוך המשתנה S את ה-gcd(7,12).

<u>רשימות:</u>

רשימה נכתבת בתוך סוגריים מרובעים, למשל [1,2,3]. רשימה ריקה []. ניתן להשתמש בסימון [H|T] על מנת לחלק את הרשימה לראש וזנב. דוגמה:

```
?- [a | [b,c,d]] = [a,b,c,d].
true.
```

למשל,

```
?- [H|T] = [a, b, c, d, e].
H = a,
T = [b, c, d, e].
```

H הוא תמיד האיבר הראשון וT הוא שאר הרשימה בלעדיו – היא תמיד רשימה בעצמה. ביצוע $[a|[b,c,d]]$ שקול להוספת איבר לראש הרשימה.

- **<u>אורך רשימה:</u>** נראה קוד בפרולוג שמקבל רשימה ומשתנה ומחזיר לתוך משתנה את מספר האיברים ברשימה.

```
size.pl
size([], 0).
size([H|T], N) :- size(T,N1), N is N1 + 1.
```

למעשה זה הוגדר מבסיס וצעד. הבסיס הוא רשימה ריקה, במקרה זה אורכה 0. אחרת, הצעד הוא שהאורך של [H|T] הוא כמו האורך של *T* ועוד 1.

- **<u>קוד בפרולוג שבודק אם איבר נמצא ברשימה נתונה:</u>** החוק הראשון הוא שX נמצא ברשימה אם הוא הראש שלה. נשים לב שX מופיע פעמיים – פעם בתור מה שמחפשים ופעם בתור הראש. החוק השני הוא שהוא נמצא ברשימה אם הוא נמצא בזנב: כלומר קוראים ברקורסיה על שאר הרשימה ובודקים אם הוא שם. נשים לב כי _ פירושו "יש כאן משהו, לא אכפת לי מהו".

```
isin.pl
isin(X, [X|_]).
isin(X, [_|T]) :- isin(X,T).
```

- **<u>קוד בפרולוג שמקבל שתי רשימות ומשתנה נוסף ומחזיר במשתנה הנוסף את שרשור שתי הרשימות:</u>** במקרה הראשון שבו הרשימה הראשונה ריקה, מחזירים את הרשימה השנייה. אחרת – לוקחים את הראש של הרשימה הראשונה ושמים אותו בראש התוצאה. ומשרשרים רקורסיבית את השאר: ללא האיבר הראשון.

```
concat([],L,L).
concat([X|L1], L2, [X|L3]) :- concat(L1,L2,L3).
```

- **<u>קוד שסוכם את הרשימה:</u>** במקרה הבסיס הרשימה הריקה. אחרת, סוכמים את זנב הרשימה ומוסיפים לN את האיבר הראשון.

```
sumlis([],0).
sumlis([H|T],N) :- sumlis(T,N1), N is N1 + H.
```

- **Main:**

בפרולוג אין באמת `main` , אבל אפשר ליצור `main` ולהריץ אותו כך:

```
swipl -q -s <file name> -g <what to run>
swipl -q -s hello.pl -g main
```

hello.pl

```
main :-
    write('hello prolog').
```